# Appendix B. Solutions to Exercises

# Chapter 3

## Exercise 3-1

Retrieve the actor ID, first name, and last name for all actors. Sort by last name and then by first name.

```
mysql> SELECT actor_id, first_name, last_name
    -> FROM actor
    -> ORDER BY 3,2;
+----------+------------+--------------+
| actor_id | first_name | last_name    |
+----------+------------+--------------+
|       58 | CHRISTIAN  | AKROYD       |
|      182 | DEBBIE     | AKROYD       |
|       92 | KIRSTEN    | AKROYD       |
|      118 | CUBA       | ALLEN        |
|      145 | KIM        | ALLEN        |
|      194 | MERYL      | ALLEN        |
...
|       13 | UMA        | WOOD         |
|       63 | CAMERON    | WRAY         |
|      111 | CAMERON    | ZELLWEGER    |
|      186 | JULIA      | ZELLWEGER    |
|       85 | MINNIE     | ZELLWEGER    |
+----------+------------+--------------+
200 rows in set (0.02 sec)
```

## Exercise 3-2

Retrieve the actor ID, first name, and last name for all actors whose last name equals `'WILLIAMS'` or `'DAVIS'`.

```
mysql> SELECT actor_id, first_name, last_name
    -> FROM actor
    -> WHERE last_name = 'WILLIAMS' OR last_name = 'DAVIS';
+----------+------------+-----------+
| actor_id | first_name | last_name |
+----------+------------+-----------+
|        4 | JENNIFER   | DAVIS     |
|      101 | SUSAN      | DAVIS     |
|      110 | SUSAN      | DAVIS     |
|       72 | SEAN       | WILLIAMS  |
|      137 | MORGAN     | WILLIAMS  |
|      172 | GROUCHO    | WILLIAMS  |
+----------+------------+-----------+
6 rows in set (0.03 sec)
```

## Exercise 3-3

Write a query against the `rental` table that returns the IDs of the customers who rented a film on July 5, 2005 (use the `rental.rental_date` column, and you can use the `date()` function to ignore the time component). Include a single row for each distinct customer ID.

```
mysql> SELECT DISTINCT customer_id
    -> FROM rental
    -> WHERE date(rental_date) = '2005-07-05';
+-------------+
| customer_id |
+-------------+
|           8 |
|          37 |
|          60 |
|         111 |
|         114 |
|         138 |
|         142 |
|         169 |
|         242 |
|         295 |
|         296 |
|         298 |
|         322 |
|         348 |
|         349 |
```

```
|       369 |
|       382 |
|       397 |
|       421 |
|       476 |
|       490 |
|       520 |
|       536 |
|       553 |
|       565 |
|       586 |
|       594 |
+-------------+
27 rows in set (0.22 sec)
```

## Exercise 3-4

Fill in the blanks (denoted by `< # >`) for this multitable query to achieve the following results:

```
mysql> SELECT c.email, r.return_date
    -> FROM customer c
    ->   INNER JOIN rental <1>
    ->   ON c.customer_id = <2>
    -> WHERE date(r.rental_date) = '2005-06-14'
    -> ORDER BY <3> <4>;
+-----------------------------------------+---------------------+
| email                                   | return_date         |
+-----------------------------------------+---------------------+
| DANIEL.CABRAL@sakilacustomer.org        | 2005-06-23 22:00:38 |
| TERRANCE.ROUSH@sakilacustomer.org       | 2005-06-23 21:53:46 |
| MIRIAM.MCKINNEY@sakilacustomer.org      | 2005-06-21 17:12:08 |
| GWENDOLYN.MAY@sakilacustomer.org        | 2005-06-20 02:40:27 |
| JEANETTE.GREENE@sakilacustomer.org      | 2005-06-19 23:26:46 |
| HERMAN.DEVORE@sakilacustomer.org        | 2005-06-19 03:20:09 |
| JEFFERY.PINSON@sakilacustomer.org       | 2005-06-18 21:37:33 |
| MATTHEW.MAHAN@sakilacustomer.org        | 2005-06-18 05:18:58 |
| MINNIE.ROMERO@sakilacustomer.org        | 2005-06-18 01:58:34 |
| SONIA.GREGORY@sakilacustomer.org        | 2005-06-17 21:44:11 |
| TERRENCE.GUNDERSON@sakilacustomer.org | 2005-06-17 05:28:35 |
| ELMER.NOE@sakilacustomer.org            | 2005-06-17 02:11:13 |
| JOYCE.EDWARDS@sakilacustomer.org        | 2005-06-16 21:00:26 |
| AMBER.DIXON@sakilacustomer.org          | 2005-06-16 04:02:56 |
| CHARLES.KOWALSKI@sakilacustomer.org     | 2005-06-16 02:26:34 |
```

```
| CATHERINE.CAMPBELL@sakilacustomer.org | 2005-06-15 20:43:03 |
+-------------------------------------+---------------------+
16 rows in set (0.03 sec)
```

<1> is replaced by `r`.

<2> is replaced by `r.customer_id`.

<3> is replaced by `2`.

<4> is replaced by `desc`.

# Chapter 4

The following subset of rows from the `payment` table are used as an example for the first two exercises:

```
+------------+-------------+--------+--------------------+
| payment_id | customer_id | amount | date(payment_date) |
+------------+-------------+--------+--------------------+
|        101 |           4 |   8.99 | 2005-08-18         |
|        102 |           4 |   1.99 | 2005-08-19         |
|        103 |           4 |   2.99 | 2005-08-20         |
|        104 |           4 |   6.99 | 2005-08-20         |
|        105 |           4 |   4.99 | 2005-08-21         |
|        106 |           4 |   2.99 | 2005-08-22         |
|        107 |           4 |   1.99 | 2005-08-23         |
|        108 |           5 |   0.99 | 2005-05-29         |
|        109 |           5 |   6.99 | 2005-05-31         |
|        110 |           5 |   1.99 | 2005-05-31         |
|        111 |           5 |   3.99 | 2005-06-15         |
|        112 |           5 |   2.99 | 2005-06-16         |
|        113 |           5 |   4.99 | 2005-06-17         |
|        114 |           5 |   2.99 | 2005-06-19         |
|        115 |           5 |   4.99 | 2005-06-20         |
|        116 |           5 |   4.99 | 2005-07-06         |
|        117 |           5 |   2.99 | 2005-07-08         |
|        118 |           5 |   4.99 | 2005-07-09         |
|        119 |           5 |   5.99 | 2005-07-09         |
|        120 |           5 |   1.99 | 2005-07-09         |
+------------+-------------+--------+--------------------+
```

## Exercise 4-1

Which of the payment IDs would be returned by the following filter conditions?

```
customer_id <> 5 AND (amount > 8 OR date(payment_date) = '2005-08-23')
```

Payment IDs 101 and 107.

## Exercise 4-2

Which of the payment IDs would be returned by the following filter conditions?

```
customer_id = 5 AND NOT (amount > 6 OR date(payment_date) = '2005-06-19')
```

Payment IDs 108, 110, 111, 112, 113, 115, 116, 117, 118, 119, and 120.

## Exercise 4-3

Construct a query that retrieves all rows from the `payment` table where the amount is either 1.98, 7.98, or 9.98.

```
mysql> SELECT amount
    -> FROM payment
    -> WHERE amount IN (1.98, 7.98, 9.98);
+--------+
| amount |
+--------+
|   7.98 |
|   9.98 |
|   1.98 |
|   7.98 |
|   7.98 |
|   7.98 |
|   7.98 |
+--------+
7 rows in set (0.01 sec)
```

## Exercise 4-4

Construct a query that finds all customers whose last name contains an *A* in the second position and a *W* anywhere after the *A*.

```
mysql> SELECT first_name, last_name
    -> FROM customer
    -> WHERE last_name LIKE '_A%W%';
+------------+------------+
| first_name | last_name  |
+------------+------------+
| KAY        | CALDWELL   |
| JOHN       | FARNSWORTH |
| JILL       | HAWKINS    |
| LEE        | HAWKS      |
| LAURIE     | LAWRENCE   |
| JEANNE     | LAWSON     |
| LAWRENCE   | LAWTON     |
| SAMUEL     | MARLOW     |
| ERICA      | MATTHEWS   |
+------------+------------+
9 rows in set (0.02 sec)
```

# Chapter 5

## Exercise 5-1

Fill in the blanks (denoted by `< # >`) for the following query to obtain the results that follow:

```
mysql> SELECT c.first_name, c.last_name, a.address, ct.city
    -> FROM customer c
    ->   INNER JOIN address <1>
    ->   ON c.address_id = a.address_id
    ->   INNER JOIN city ct
    ->   ON a.city_id = <2>
    -> WHERE a.district = 'California';
+------------+-----------+-----------------------+---------------+
| first_name | last_name | address               | city          |
+------------+-----------+-----------------------+---------------+
| PATRICIA   | JOHNSON   | 1121 Loja Avenue      | San Bernardino |
| BETTY      | WHITE     | 770 Bydgoszcz Avenue  | Citrus Heights |
```

```
| ALICE     | STEWART   | 1135 Izumisano Parkway | Fontana      |
| ROSA      | REYNOLDS  | 793 Cam Ranh Avenue    | Lancaster    |
| RENEE     | LANE      | 533 al-Ayn Boulevard   | Compton      |
| KRISTIN   | JOHNSTON  | 226 Brest Manor        | Sunnyvale    |
| CASSANDRA | WALTERS   | 920 Kumbakonam Loop    | Salinas      |
| JACOB     | LANCE     | 1866 al-Qatif Avenue   | El Monte     |
| RENE      | MCALISTER | 1895 Zhezqazghan Drive | Garden Grove |
+-----------+-----------+------------------------+--------------+
9 rows in set (0.00 sec)
```

<1> is replaced by `a`.

<2> is replaced by `ct.city_id`.

## Exercise 5-2

Write a query that returns the title of every film in which an actor with the first name JOHN appeared.

```
mysql> SELECT f.title
    -> FROM film f
    ->   INNER JOIN film_actor fa
    ->   ON f.film_id = fa.film_id
    ->   INNER JOIN actor a
    ->   ON fa.actor_id = a.actor_id
    -> WHERE a.first_name = 'JOHN';
+--------------------------+
| title                    |
+--------------------------+
| ALLEY EVOLUTION          |
| BEVERLY OUTLAW           |
| CANDLES GRAPES           |
| CLEOPATRA DEVIL          |
| COLOR PHILADELPHIA       |
| CONQUERER NUTS           |
| DAUGHTER MADIGAN         |
| GLEAMING JAWBREAKER      |
| GOLDMINE TYCOON          |
| HOME PITY                |
| INTERVIEW LIAISONS       |
| ISHTAR ROCKETEER         |
| JAPANESE RUN             |
| JERSEY SASSY             |
| LUKE MUMMY               |
```

```
| MILLION ACE              |
| MONSTER SPARTACUS        |
| NAME DETECTIVE           |
| NECKLACE OUTBREAK        |
| NEWSIES STORY            |
| PET HAUNTING             |
| PIANIST OUTFIELD         |
| PINOCCHIO SIMON          |
| PITTSBURGH HUNCHBACK     |
| QUILLS BULL              |
| RAGING AIRPLANE          |
| ROXANNE REBEL            |
| SATISFACTION CONFIDENTIAL |
| SONG HEDWIG              |
+--------------------------+
29 rows in set (0.07 sec)
```

## Exercise 5-3

Construct a query that returns all addresses that are in the same city. You will need to join the address table to itself, and each row should include two different addresses.

```
mysql> SELECT a1.address addr1, a2.address addr2, a1.city_id
    -> FROM address a1
    ->   INNER JOIN address a2
    -> ON a1.city_id = a2.city_id
    ->   AND a1.address < a2.address;
+---------------------+-------------------+---------+
| addr1               | addr2             | city_id |
+---------------------+-------------------+---------+
| 23 Workhaven Lane   | 47 MySakila Drive |     300 |
| 1411 Lillydale Drive | 28 MySQL Boulevard |     576 |
| 1497 Yuzhou Drive   | 548 Uruapan Street |     312 |
| 43 Vilnius Manor    | 587 Benguela Manor |      42 |
+---------------------+-------------------+---------+
4 rows in set (0.01 sec)
```

# Chapter 6

## Exercise 6-1

If set A = {L M N O P} and set B = {P Q R S T}, what sets are generated by the following operations?

- `A union B`
- `A union all B`
- `A intersect B`
- `A except B`
    1. `A union B` = {L M N O P Q R S T}
    2. `A union all B` = {L M N O P P Q R S T}
    3. `A intersect B` = {P}
    4. `A except B` = {L M N O}

## Exercise 6-2

Write a compound query that finds the first and last names of all actors and customers whose last name starts with L.

```
mysql> SELECT first_name, last_name
    -> FROM actor
    -> WHERE last_name LIKE 'L%'
    -> UNION
    -> SELECT first_name, last_name
    -> FROM customer
    -> WHERE last_name LIKE 'L%';
+------------+--------------+
| first_name | last_name    |
+------------+--------------+
| MATTHEW    | LEIGH        |
| JOHNNY     | LOLLOBRIGIDA |
| MISTY      | LAMBERT      |
| JACOB      | LANCE        |
| RENEE      | LANE         |
| HEIDI      | LARSON       |
| DARYL      | LARUE        |
| LAURIE     | LAWRENCE     |
| JEANNE     | LAWSON       |
| LAWRENCE   | LAWTON       |
| KIMBERLY   | LEE          |
```

```
| LOUIS     | LEONE       |
| SARAH     | LEWIS       |
| GEORGE    | LINTON      |
| MAUREEN   | LITTLE      |
| DWIGHT    | LOMBARDI    |
| JACQUELINE | LONG       |
| AMY       | LOPEZ       |
| BARRY     | LOVELACE    |
| PRISCILLA | LOWE        |
| VELMA     | LUCAS       |
| WILLARD   | LUMPKIN     |
| LEWIS     | LYMAN       |
| JACKIE    | LYNCH       |
+-----------+-------------+
24 rows in set (0.01 sec)
```

## Exercise 6-3

Sort the results from Exercise 6-2 by the `last_name` column.

```
mysql> SELECT first_name, last_name
    -> FROM actor
    -> WHERE last_name LIKE 'L%'
    -> UNION
    -> SELECT first_name, last_name
    -> FROM customer
    -> WHERE last_name LIKE 'L%'
    -> ORDER BY last_name;
+-----------+-------------+
| first_name | last_name   |
+-----------+-------------+
| MISTY     | LAMBERT     |
| JACOB     | LANCE       |
| RENEE     | LANE        |
| HEIDI     | LARSON      |
| DARYL     | LARUE       |
| LAURIE    | LAWRENCE    |
| JEANNE    | LAWSON      |
| LAWRENCE  | LAWTON      |
| KIMBERLY  | LEE         |
| MATTHEW   | LEIGH       |
| LOUIS     | LEONE       |
| SARAH     | LEWIS       |
| GEORGE    | LINTON      |
```

```
| MAUREEN    | LITTLE      |
| JOHNNY     | LOLLOBRIGIDA |
| DWIGHT     | LOMBARDI    |
| JACQUELINE | LONG        |
| AMY        | LOPEZ       |
| BARRY      | LOVELACE    |
| PRISCILLA  | LOWE        |
| VELMA      | LUCAS       |
| WILLARD    | LUMPKIN     |
| LEWIS      | LYMAN       |
| JACKIE     | LYNCH       |
+------------+-------------+
24 rows in set (0.00 sec)
```

# Chapter 7

## Exercise 7-1

Write a query that returns the 17th through 25th characters of the string `'Please find the substring in this string'`.

```
mysql> SELECT SUBSTRING('Please find the substring in this string',17,9);
+-----------------------------------------------------------+
| SUBSTRING('Please find the substring in this string',17,9) |
+-----------------------------------------------------------+
| substring                                                 |
+-----------------------------------------------------------+
1 row in set (0.00 sec)
```

## Exercise 7-2

Write a query that returns the absolute value and sign ( `-1`, `0`, or `1`) of the number `-25.76823`. Also return the number rounded to the nearest hundredth.

```
mysql> SELECT ABS(-25.76823), SIGN(-25.76823), ROUND(-25.76823, 2);
+----------------+-----------------+---------------------+
| ABS(-25.76823) | SIGN(-25.76823) | ROUND(-25.76823, 2) |
+----------------+-----------------+---------------------+
|       25.76823 |              -1 |              -25.77 |
+----------------+-----------------+---------------------+
1 row in set (0.00 sec)
```

## Exercise 7-3

Write a query to return just the month portion of the current date.

```
mysql> SELECT EXTRACT(MONTH FROM CURRENT_DATE());
+---------------------------------+
| EXTRACT(MONTH FROM CURRENT_DATE) |
+---------------------------------+
|                              12 |
+---------------------------------+
1 row in set (0.02 sec)
```

(Your result will most likely be different, unless it happens to be December when you try this exercise.)

# Chapter 8

## Exercise 8-1

Construct a query that counts the number of rows in the `payment` table.

```
mysql> SELECT count(*) FROM payment;
+----------+
| count(*) |
+----------+
|   16049 |
+----------+
1 row in set (0.02 sec)
```

## Exercise 8-2

Modify your query from Exercise 8-1 to count the number of payments made by each customer. Show the customer ID and the total amount paid for each customer.

```
mysql> SELECT customer_id, count(*), sum(amount)
    -> FROM payment
    -> GROUP BY customer_id;
+-------------+----------+-------------+
| customer_id | count(*) | sum(amount) |
```

```
+-------------+----------+-------------+
|           1 |       32 |      118.68 |
|           2 |       27 |      128.73 |
|           3 |       26 |      135.74 |
|           4 |       22 |       81.78 |
|           5 |       38 |      144.62 |
...
|         595 |       30 |      117.70 |
|         596 |       28 |       96.72 |
|         597 |       25 |       99.75 |
|         598 |       22 |       83.78 |
|         599 |       19 |       83.81 |
+-------------+----------+-------------+
599 rows in set (0.03 sec)
```

## Exercise 8-3

Modify your query from Exercise 8-2 to include only those customers who have made at least 40 payments.

```
mysql> SELECT customer_id, count(*), sum(amount)
    -> FROM payment
    -> GROUP BY customer_id
    -> HAVING count(*) >= 40;
+-------------+----------+-------------+
| customer_id | count(*) | sum(amount) |
+-------------+----------+-------------+
|          75 |       41 |      155.59 |
|         144 |       42 |      195.58 |
|         148 |       46 |      216.54 |
|         197 |       40 |      154.60 |
|         236 |       42 |      175.58 |
|         469 |       40 |      177.60 |
|         526 |       45 |      221.55 |
+-------------+----------+-------------+
7 rows in set (0.03 sec)
```

# Chapter 9

## Exercise 9-1

Construct a query against the `film` table that uses a filter condition with a noncorrelated subquery against the `category` table to find all action films ( `category.name = 'Action'` ).

```
mysql> SELECT title
    -> FROM film
    -> WHERE film_id IN
    ->  (SELECT fc.film_id
    ->   FROM film_category fc INNER JOIN category c
    ->     ON fc.category_id = c.category_id
    ->   WHERE c.name = 'Action');
+------------------------+
| title                  |
+------------------------+
| AMADEUS HOLY           |
| AMERICAN CIRCUS        |
| ANTITRUST TOMATOES     |
| ARK RIDGEMONT          |
| BAREFOOT MANCHURIAN    |
| BERETS AGENT           |
| BRIDE INTRIGUE         |
| BULL SHAWSHANK         |
| CADDYSHACK JEDI        |
| CAMPUS REMEMBER        |
| CASUALTIES ENCINO      |
| CELEBRITY HORN         |
| CLUELESS BUCKET        |
| CROW GREASE            |
| DANCES NONE            |
| DARKO DORADO           |
| DARN FORRESTER         |
| DEVIL DESIRE           |
| DRAGON SQUAD           |
| DREAM PICKUP           |
| DRIFTER COMMANDMENTS   |
| EASY GLADIATOR         |
| ENTRAPMENT SATISFACTION |
| EXCITEMENT EVE         |
| FANTASY TROOPERS       |
| FIREHOUSE VIETNAM      |
| FOOL MOCKINGBIRD       |
```

```
| FORREST SONS          |
| GLASS DYING           |
| GOSFORD DONNIE        |
| GRAIL FRANKENSTEIN    |
| HANDICAP BOONDOCK     |
| HILLS NEIGHBORS       |
| KISSING DOLLS         |
| LAWRENCE LOVE         |
| LORD ARIZONA          |
| LUST LOCK             |
| MAGNOLIA FORRESTER    |
| MIDNIGHT WESTWARD     |
| MINDS TRUMAN          |
| MOCKINGBIRD HOLLYWOOD |
| MONTEZUMA COMMAND     |
| PARK CITIZEN          |
| PATRIOT ROMAN         |
| PRIMARY GLASS         |
| QUEST MUSSOLINI       |
| REAR TRADING          |
| RINGS HEARTBREAKERS   |
| RUGRATS SHAKESPEARE   |
| SHRUNK DIVINE         |
| SIDE ARK              |
| SKY MIRACLE           |
| SOUTH WAIT            |
| SPEAKEASY DATE        |
| STAGECOACH ARMAGEDDON |
| STORY SIDE            |
| SUSPECTS QUILLS       |
| TRIP NEWTON           |
| TRUMAN CRAZY          |
| UPRISING UPTOWN       |
| WATERFRONT DELIVERANCE |
| WEREWOLF LOLA         |
| WOMEN DORADO          |
| WORST BANGER          |
+-----------------------+
64 rows in set (0.06 sec)
```

## Exercise 9-2

Rework the query from Exercise 9-1 using a *correlated* subquery against the `category` and `film_category` tables to achieve the same results.

```
mysql> SELECT f.title
    -> FROM film f
    -> WHERE EXISTS
    ->  (SELECT 1
    ->   FROM film_category fc INNER JOIN category c
    ->     ON fc.category_id = c.category_id
    ->   WHERE c.name = 'Action'
    ->     AND fc.film_id = f.film_id);
+------------------------+
| title                  |
+------------------------+
| AMADEUS HOLY           |
| AMERICAN CIRCUS        |
| ANTITRUST TOMATOES     |
| ARK RIDGEMONT          |
| BAREFOOT MANCHURIAN    |
| BERETS AGENT           |
| BRIDE INTRIGUE         |
| BULL SHAWSHANK         |
| CADDYSHACK JEDI        |
| CAMPUS REMEMBER        |
| CASUALTIES ENCINO      |
| CELEBRITY HORN         |
| CLUELESS BUCKET        |
| CROW GREASE            |
| DANCES NONE            |
| DARKO DORADO           |
| DARN FORRESTER         |
| DEVIL DESIRE           |
| DRAGON SQUAD           |
| DREAM PICKUP           |
| DRIFTER COMMANDMENTS   |
| EASY GLADIATOR         |
| ENTRAPMENT SATISFACTION |
| EXCITEMENT EVE         |
| FANTASY TROOPERS       |
| FIREHOUSE VIETNAM      |
| FOOL MOCKINGBIRD       |
| FORREST SONS           |
| GLASS DYING            |
| GOSFORD DONNIE         |
| GRAIL FRANKENSTEIN     |
| HANDICAP BOONDOCK      |
| HILLS NEIGHBORS        |
| KISSING DOLLS          |
| LAWRENCE LOVE          |
```

```
| LORD ARIZONA          |
| LUST LOCK             |
| MAGNOLIA FORRESTER    |
| MIDNIGHT WESTWARD     |
| MINDS TRUMAN          |
| MOCKINGBIRD HOLLYWOOD |
| MONTEZUMA COMMAND     |
| PARK CITIZEN          |
| PATRIOT ROMAN         |
| PRIMARY GLASS         |
| QUEST MUSSOLINI       |
| REAR TRADING          |
| RINGS HEARTBREAKERS   |
| RUGRATS SHAKESPEARE   |
| SHRUNK DIVINE         |
| SIDE ARK              |
| SKY MIRACLE           |
| SOUTH WAIT            |
| SPEAKEASY DATE        |
| STAGECOACH ARMAGEDDON |
| STORY SIDE            |
| SUSPECTS QUILLS       |
| TRIP NEWTON           |
| TRUMAN CRAZY          |
| UPRISING UPTOWN       |
| WATERFRONT DELIVERANCE |
| WEREWOLF LOLA         |
| WOMEN DORADO          |
| WORST BANGER          |
+-----------------------+
64 rows in set (0.02 sec)
```

## Exercise 9-3

Join the following query to a subquery against the `film_actor` table to show the level of each actor:

```
SELECT 'Hollywood Star' level, 30 min_roles, 99999 max_roles
UNION ALL
SELECT 'Prolific Actor' level, 20 min_roles, 29 max_roles
UNION ALL
SELECT 'Newcomer' level, 1 min_roles, 19 max_roles
```

The subquery against the `film_actor` table should count the number of rows for each actor using `group by actor_id`, and the count should be compared to the `min_roles` / `max_roles` columns to determine which level each actor belongs to.

```
mysql> SELECT actr.actor_id, grps.level
    -> FROM
    ->  (SELECT actor_id, count(*) num_roles
    ->   FROM film_actor
    ->   GROUP BY actor_id
    ->  ) actr
    ->   INNER JOIN
    ->  (SELECT 'Hollywood Star' level, 30 min_roles, 99999 max_roles
    ->   UNION ALL
    ->   SELECT 'Prolific Actor' level, 20 min_roles, 29 max_roles
    ->   UNION ALL
    ->   SELECT 'Newcomer' level, 1 min_roles, 19 max_roles
    ->  ) grps
    ->   ON actr.num_roles BETWEEN grps.min_roles AND grps.max_roles;
+----------+---------------+
| actor_id | level         |
+----------+---------------+
|        1 | Newcomer      |
|        2 | Prolific Actor |
|        3 | Prolific Actor |
|        4 | Prolific Actor |
|        5 | Prolific Actor |
|        6 | Prolific Actor |
|        7 | Hollywood Star |
...
|      195 | Prolific Actor |
|      196 | Hollywood Star |
|      197 | Hollywood Star |
|      198 | Hollywood Star |
|      199 | Newcomer      |
|      200 | Prolific Actor |
+----------+---------------+
200 rows in set (0.03 sec)
```

# Chapter 10

## Exercise 10-1

Using the following table definitions and data, write a query that returns each customer name along with their total payments:

```
                Customer:
Customer_id  Name
-----------     ---------------
1            John Smith
2            Kathy Jones
3            Greg Oliver

                Payment:
Payment_id   Customer_id  Amount
----------      -----------     --------
101          1           8.99
102          3           4.99
103          1           7.99
```

Include all customers, even if no payment records exist for that customer.

```
mysql> SELECT c.name, sum(p.amount)
    -> FROM customer c LEFT OUTER JOIN payment p
    ->   ON c.customer_id = p.customer_id
    -> GROUP BY c.name;
+-------------+---------------+
| name        | sum(p.amount) |
+-------------+---------------+
| John Smith  |         16.98 |
| Kathy Jones |          NULL |
| Greg Oliver |          4.99 |
+-------------+---------------+
3 rows in set (0.00 sec)
```

## Exercise 10-2

Reformulate your query from Exercise 10-1 to use the other outer join type (e.g., if you used a left outer join in Exercise 10-1, use a right outer join this time) such that the results are identical to Exercise 10-1.

```
MySQL> SELECT c.name, sum(p.amount)
    -> FROM payment p RIGHT OUTER JOIN customer c
    ->   ON c.customer_id = p.customer_id
    -> GROUP BY c.name;
+-------------+---------------+
| name        | sum(p.amount) |
+-------------+---------------+
| John Smith  |         16.98 |
| Kathy Jones |          NULL |
| Greg Oliver |          4.99 |
+-------------+---------------+
3 rows in set (0.00 sec)
```

## Exercise 10-3 (Extra Credit)

Devise a query that will generate the set {1, 2, 3, ..., 99, 100}. (Hint: use a cross join with at least two `from` clause subqueries.)

```
SELECT ones.x + tens.x + 1
FROM
 (SELECT 0 x UNION ALL
  SELECT 1 x UNION ALL
  SELECT 2 x UNION ALL
  SELECT 3 x UNION ALL
  SELECT 4 x UNION ALL
  SELECT 5 x UNION ALL
  SELECT 6 x UNION ALL
  SELECT 7 x UNION ALL
  SELECT 8 x UNION ALL
  SELECT 9 x
 ) ones
  CROSS JOIN
 (SELECT 0 x UNION ALL
  SELECT 10 x UNION ALL
  SELECT 20 x UNION ALL
  SELECT 30 x UNION ALL
  SELECT 40 x UNION ALL
  SELECT 50 x UNION ALL
  SELECT 60 x UNION ALL
  SELECT 70 x UNION ALL
  SELECT 80 x UNION ALL
  SELECT 90 x
 ) tens;
```

# Chapter 11

## Exercise 11-1

Rewrite the following query, which uses a simple `case` expression, so that the same results are achieved using a searched `case` expression. Try to use as few `when` clauses as possible.

```
SELECT name,
  CASE name
    WHEN 'English' THEN 'latin1'
    WHEN 'Italian' THEN 'latin1'
    WHEN 'French' THEN 'latin1'
    WHEN 'German' THEN 'latin1'
    WHEN 'Japanese' THEN 'utf8'
    WHEN 'Mandarin' THEN 'utf8'
    ELSE 'Unknown'
  END character_set
FROM language;
```

```
SELECT name,
  CASE
    WHEN name IN ('English','Italian','French','German')
      THEN 'latin1'
    WHEN name IN ('Japanese','Mandarin')
      THEN 'utf8'
    ELSE 'Unknown'
  END character_set
FROM language;
```

## Exercise 11-2

Rewrite the following query so that the result set contains a single row with five columns (one for each rating). Name the five columns `G`, `PG`, `PG_13`, `R`, and `NC_17`.

```
mysql> SELECT rating, count(*)
    -> FROM film
    -> GROUP BY rating;
+--------+----------+
| rating | count(*) |
```

```
+--------+----------+
| PG     |      194 |
| G      |      178 |
| NC-17  |      210 |
| PG-13  |      223 |
| R      |      195 |
+--------+----------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT
    ->   sum(CASE WHEN rating = 'G' THEN 1 ELSE 0 END) g,
    ->   sum(CASE WHEN rating = 'PG' THEN 1 ELSE 0 END) pg,
    ->   sum(CASE WHEN rating = 'PG-13' THEN 1 ELSE 0 END) pg_13,
    ->   sum(CASE WHEN rating = 'R' THEN 1 ELSE 0 END) r,
    ->   sum(CASE WHEN rating = 'NC-17' THEN 1 ELSE 0 END) nc_17
    -> FROM film;
+------+------+-------+------+-------+
| g    | pg   | pg_13 | r    | nc_17 |
+------+------+-------+------+-------+
|  178 |  194 |   223 |  195 |   210 |
+------+------+-------+------+-------+
1 row in set (0.00 sec)
```

# Chapter 12

## Exercise 12-1

Generate a unit of work to transfer $50 from account 123 to account 789. You will need to insert two rows into the `transaction` table and update two rows in the `account` table. Use the following table definitions/data:

```
                Account:
account_id      avail_balance last_activity_date
----------      -------------  ------------------
123         500       2019-07-10 20:53:27
789         75        2019-06-22 15:18:35

                Transaction:
txn_id          txn_date account_id     txn_type_cd   amount
--------- ------------   -----------     -----------     --------
1001            2019-05-15   123        C           500
```

```
1002        2019-06-01  789      C          75
```

Use `txn_type_cd = 'C'` to indicate a credit (addition), and use `txn_type_cd = 'D'` to indicate a debit (subtraction).

```
START TRANSACTION;

INSERT INTO transaction
 (txn_id, txn_date, account_id, txn_type_cd, amount)
VALUES
 (1003, now(), 123, 'D', 50);

INSERT INTO transaction
 (txn_id, txn_date, account_id, txn_type_cd, amount)
VALUES
 (1004, now(), 789, 'C', 50);

UPDATE account
SET avail_balance = available_balance - 50,
  last_activity_date = now()
WHERE account_id = 123;

UPDATE account
SET avail_balance = available_balance + 50,
  last_activity_date = now()
WHERE account_id = 789;

COMMIT;
```

# Chapter 13

## Exercise 13-1

Generate an `alter table` statement for the `rental` table so that an error will be raised if a row having a value found in the `rental.customer_id` column is deleted from the `customer` table.

```
ALTER TABLE rental
ADD CONSTRAINT fk_rental_customer_id FOREIGN KEY (customer_id)
REFERENCES customer (customer_id) ON DELETE RESTRICT;
```

## Exercise 13-2

Generate a multicolumn index on the `payment` table that could be used by both of the following queries:

```
SELECT customer_id, payment_date, amount
FROM payment
WHERE payment_date > cast('2019-12-31 23:59:59' as datetime);

SELECT customer_id, payment_date, amount
FROM payment
WHERE payment_date > cast('2019-12-31 23:59:59' as datetime)
  AND amount < 5;
```

```
CREATE INDEX idx_payment01
ON payment (payment_date, amount);
```

# Chapter 14

## Exercise 14-1

Create a view definition that can be used by the following query to generate the given results:

```
SELECT title, category_name, first_name, last_name
FROM film_ctgry_actor
WHERE last_name = 'FAWCETT';

+--------------------+---------------+------------+-----------+
| title              | category_name | first_name | last_name |
+--------------------+---------------+------------+-----------+
| ACE GOLDFINGER     | Horror        | BOB        | FAWCETT   |
| ADAPTATION HOLES   | Documentary   | BOB        | FAWCETT   |
| CHINATOWN GLADIATOR | New          | BOB        | FAWCETT   |
| CIRCUS YOUTH       | Children      | BOB        | FAWCETT   |
| CONTROL ANTHEM     | Comedy        | BOB        | FAWCETT   |
| DARES PLUTO        | Animation     | BOB        | FAWCETT   |
| DARN FORRESTER     | Action        | BOB        | FAWCETT   |
| DAZED PUNK         | Games         | BOB        | FAWCETT   |
| DYNAMITE TARZAN    | Classics      | BOB        | FAWCETT   |
| HATE HANDICAP      | Comedy        | BOB        | FAWCETT   |
```

```
| HOMICIDE PEACH      | Family       | BOB       | FAWCETT   |
| JACKET FRISCO       | Drama        | BOB       | FAWCETT   |
| JUMANJI BLADE       | New          | BOB       | FAWCETT   |
| LAWLESS VISION      | Animation    | BOB       | FAWCETT   |
| LEATHERNECKS DWARFS | Travel       | BOB       | FAWCETT   |
| OSCAR GOLD          | Animation    | BOB       | FAWCETT   |
| PELICAN COMFORTS    | Documentary  | BOB       | FAWCETT   |
| PERSONAL LADYBUGS   | Music        | BOB       | FAWCETT   |
| RAGING AIRPLANE     | Sci-Fi       | BOB       | FAWCETT   |
| RUN PACIFIC         | New          | BOB       | FAWCETT   |
| RUNNER MADIGAN      | Music        | BOB       | FAWCETT   |
| SADDLE ANTITRUST    | Comedy       | BOB       | FAWCETT   |
| SCORPION APOLLO     | Drama        | BOB       | FAWCETT   |
| SHAWSHANK BUBBLE    | Travel       | BOB       | FAWCETT   |
| TAXI KICK           | Music        | BOB       | FAWCETT   |
| BERETS AGENT        | Action       | JULIA     | FAWCETT   |
| BOILED DARES        | Travel       | JULIA     | FAWCETT   |
| CHISUM BEHAVIOR     | Family       | JULIA     | FAWCETT   |
| CLOSER BANG         | Comedy       | JULIA     | FAWCETT   |
| DAY UNFAITHFUL      | New          | JULIA     | FAWCETT   |
| HOPE TOOTSIE        | Classics     | JULIA     | FAWCETT   |
| LUKE MUMMY          | Animation    | JULIA     | FAWCETT   |
| MULAN MOON          | Comedy       | JULIA     | FAWCETT   |
| OPUS ICE            | Foreign      | JULIA     | FAWCETT   |
| POLLOCK DELIVERANCE | Foreign      | JULIA     | FAWCETT   |
| RIDGEMONT SUBMARINE | New          | JULIA     | FAWCETT   |
| SHANGHAI TYCOON     | Travel       | JULIA     | FAWCETT   |
| SHAWSHANK BUBBLE    | Travel       | JULIA     | FAWCETT   |
| THEORY MERMAID      | Animation    | JULIA     | FAWCETT   |
| WAIT CIDER          | Animation    | JULIA     | FAWCETT   |
+---------------------+--------------+-----------+-----------+
40 rows in set (0.00 sec)
```

```
CREATE VIEW film_ctgry_actor
AS
SELECT f.title,
  c.name category_name,
  a.first_name,
  a.last_name
FROM film f
  INNER JOIN film_category fc
  ON f.film_id = fc.film_id
  INNER JOIN category c
  ON fc.category_id = c.category_id
```

```
  INNER JOIN film_actor fa
  ON fa.film_id = f.film_id
  INNER JOIN actor a
  ON fa.actor_id = a.actor_id;
```

## Exercise 14-2

The film rental company manager would like to have a report that includes the name of every country, along with the total payments for all customers who live in each country. Generate a view definition that queries the `country` table and uses a scalar subquery to calculate a value for a column named `tot_payments`.

```
CREATE VIEW country_payments
AS
SELECT c.country,
 (SELECT sum(p.amount)
  FROM city ct
    INNER JOIN address a
    ON ct.city_id = a.city_id
    INNER JOIN customer cst
    ON a.address_id = cst.address_id
    INNER JOIN payment p
    ON cst.customer_id = p.customer_id
  WHERE ct.country_id = c.country_id
 ) tot_payments
FROM country c
```

# Chapter 15

## Exercise 15-1

Write a query that lists all of the indexes in the Sakila schema. Include the table names.

```
mysql> SELECT DISTINCT table_name, index_name
    -> FROM information_schema.statistics
    -> WHERE table_schema = 'sakila';
+---------------+-----------------------------+
| TABLE_NAME    | INDEX_NAME                  |
+---------------+-----------------------------+
| actor         | PRIMARY                     |
| actor         | idx_actor_last_name         |
```

```
| address      | PRIMARY                  |
| address      | idx_fk_city_id           |
| address      | idx_location             |
| category     | PRIMARY                  |
| city         | PRIMARY                  |
| city         | idx_fk_country_id        |
| country      | PRIMARY                  |
| film         | PRIMARY                  |
| film         | idx_title                |
| film         | idx_fk_language_id       |
| film         | idx_fk_original_language_id |
| film_actor   | PRIMARY                  |
| film_actor   | idx_fk_film_id           |
| film_category | PRIMARY                 |
| film_category | fk_film_category_category |
| film_text    | PRIMARY                  |
| film_text    | idx_title_description    |
| inventory    | PRIMARY                  |
| inventory    | idx_fk_film_id           |
| inventory    | idx_store_id_film_id     |
| language     | PRIMARY                  |
| staff        | PRIMARY                  |
| staff        | idx_fk_store_id          |
| staff        | idx_fk_address_id        |
| store        | PRIMARY                  |
| store        | idx_unique_manager       |
| store        | idx_fk_address_id        |
| customer     | PRIMARY                  |
| customer     | idx_email                |
| customer     | idx_fk_store_id          |
| customer     | idx_fk_address_id        |
| customer     | idx_last_name            |
| customer     | idx_full_name            |
| rental       | PRIMARY                  |
| rental       | rental_date              |
| rental       | idx_fk_inventory_id      |
| rental       | idx_fk_customer_id       |
| rental       | idx_fk_staff_id          |
| payment      | PRIMARY                  |
| payment      | idx_fk_staff_id          |
| payment      | idx_fk_customer_id       |
| payment      | fk_payment_rental        |
| payment      | idx_payment01            |
+--------------+--------------------------+
45 rows in set (0.00 sec)
```

## Exercise 15-2

Write a query that generates output that can be used to create all of the indexes on the `sakila.customer` table. Output should be of the form:

```
"ALTER TABLE <table_name> ADD INDEX <index_name> (<column_list>)"
```

Here's one solution utilizing a `with` clause:

```
mysql> WITH idx_info AS
    ->  (SELECT s1.table_name, s1.index_name,
    ->    s1.column_name, s1.seq_in_index,
    ->    (SELECT max(s2.seq_in_index)
    ->     FROM information_schema.statistics s2
    ->     WHERE s2.table_schema = s1.table_schema
    ->       AND s2.table_name = s1.table_name
    ->       AND s2.index_name = s1.index_name) num_columns
    ->   FROM information_schema.statistics s1
    ->   WHERE s1.table_schema = 'sakila'
    ->     AND s1.table_name = 'customer'
    ->  )
    -> SELECT concat(
    ->   CASE
    ->     WHEN seq_in_index = 1 THEN
    ->       concat('ALTER TABLE ', table_name, ' ADD INDEX ',
    ->            index_name, ' (', column_name)
    ->     ELSE concat('  , ', column_name)
    ->   END,
    ->   CASE
    ->     WHEN seq_in_index = num_columns THEN ');'
    ->     ELSE ''
    ->   END
    ->  ) index_creation_statement
    -> FROM idx_info
    -> ORDER BY index_name, seq_in_index;
+-------------------------------------------------------------+
| index_creation_statement                                    |
+-------------------------------------------------------------+
| ALTER TABLE customer ADD INDEX idx_email (email);           |
| ALTER TABLE customer ADD INDEX idx_fk_address_id (address_id); |
| ALTER TABLE customer ADD INDEX idx_fk_store_id (store_id);  |
| ALTER TABLE customer ADD INDEX idx_full_name (last_name     |
|   , first_name);                                            |
| ALTER TABLE customer ADD INDEX idx_last_name (last_name);   |
```

```
| ALTER TABLE customer ADD INDEX PRIMARY (customer_id);         |
+---------------------------------------------------------------+
7 rows in set (0.00 sec)
```

After reading Chapter 16, however, you could use the following:

```
mysql> SELECT concat('ALTER TABLE ', table_name, ' ADD INDEX ',
    ->   index_name, ' (',
    ->   group_concat(column_name order by seq_in_index separator ', '),
    ->   ');'
    ->  ) index_creation_statement
    -> FROM information_schema.statistics
    -> WHERE table_schema = 'sakila'
    ->   AND table_name = 'customer'
    -> GROUP BY table_name, index_name;
+-------------------------------------------------------------------------+
| index_creation_statement                                                |
+-------------------------------------------------------------------------+
| ALTER TABLE customer ADD INDEX idx_email (email);                       |
| ALTER TABLE customer ADD INDEX idx_fk_address_id (address_id);          |
| ALTER TABLE customer ADD INDEX idx_fk_store_id (store_id);              |
| ALTER TABLE customer ADD INDEX idx_full_name (last_name, first_name); |
| ALTER TABLE customer ADD INDEX idx_last_name (last_name);               |
| ALTER TABLE customer ADD INDEX PRIMARY (customer_id);                   |
+-------------------------------------------------------------------------+
6 rows in set (0.00 sec)
```

# Chapter 16

For all exercises in this section, use the following data set from the `Sales_Fact` table:

```
Sales_Fact
+---------+----------+-----------+
| year_no | month_no | tot_sales |
+---------+----------+-----------+
|    2019 |        1 |     19228 |
|    2019 |        2 |     18554 |
|    2019 |        3 |     17325 |
|    2019 |        4 |     13221 |
|    2019 |        5 |      9964 |
|    2019 |        6 |     12658 |
|    2019 |        7 |     14233 |
```

```
|   2019 |       8 |    17342 |
|   2019 |       9 |    16853 |
|   2019 |      10 |    17121 |
|   2019 |      11 |    19095 |
|   2019 |      12 |    21436 |
|   2020 |       1 |    20347 |
|   2020 |       2 |    17434 |
|   2020 |       3 |    16225 |
|   2020 |       4 |    13853 |
|   2020 |       5 |    14589 |
|   2020 |       6 |    13248 |
|   2020 |       7 |     8728 |
|   2020 |       8 |     9378 |
|   2020 |       9 |    11467 |
|   2020 |      10 |    13842 |
|   2020 |      11 |    15742 |
|   2020 |      12 |    18636 |
+---------+----------+-----------+
24 rows in set (0.00 sec)
```

## Exercise 16-1

Write a query that retrieves every row from `Sales_Fact`, and add a column to generate a ranking based on the `tot_sales` column values. The highest value should receive a ranking of 1, and the lowest a ranking of 24.

```
mysql> SELECT year_no, month_no, tot_sales,
    ->   rank() over (order by tot_sales desc) sales_rank
    -> FROM sales_fact;
+---------+----------+-----------+------------+
| year_no | month_no | tot_sales | sales_rank |
+---------+----------+-----------+------------+
|   2019 |      12 |    21436 |        1 |
|   2020 |       1 |    20347 |        2 |
|   2019 |       1 |    19228 |        3 |
|   2019 |      11 |    19095 |        4 |
|   2020 |      12 |    18636 |        5 |
|   2019 |       2 |    18554 |        6 |
|   2020 |       2 |    17434 |        7 |
|   2019 |       8 |    17342 |        8 |
|   2019 |       3 |    17325 |        9 |
|   2019 |      10 |    17121 |       10 |
|   2019 |       9 |    16853 |       11 |
|   2020 |       3 |    16225 |       12 |
```

```
|   2020 |     11 |    15742 |      13 |
|   2020 |      5 |    14589 |      14 |
|   2019 |      7 |    14233 |      15 |
|   2020 |      4 |    13853 |      16 |
|   2020 |     10 |    13842 |      17 |
|   2020 |      6 |    13248 |      18 |
|   2019 |      4 |    13221 |      19 |
|   2019 |      6 |    12658 |      20 |
|   2020 |      9 |    11467 |      21 |
|   2019 |      5 |     9964 |      22 |
|   2020 |      8 |     9378 |      23 |
|   2020 |      7 |     8728 |      24 |
+---------+----------+-----------+------------+
24 rows in set (0.02 sec)
```

## Exercise 16-2

Modify the query from the previous exercise to generate two sets of rankings from 1 to 12, one for 2019 data and one for 2020.

```
mysql> SELECT year_no, month_no, tot_sales,
    ->   rank() over (partition by year_no
    ->                order by tot_sales desc) sales_rank
    -> FROM sales_fact;
+---------+----------+-----------+------------+
| year_no | month_no | tot_sales | sales_rank |
+---------+----------+-----------+------------+
|    2019 |       12 |     21436 |          1 |
|    2019 |        1 |     19228 |          2 |
|    2019 |       11 |     19095 |          3 |
|    2019 |        2 |     18554 |          4 |
|    2019 |        8 |     17342 |          5 |
|    2019 |        3 |     17325 |          6 |
|    2019 |       10 |     17121 |          7 |
|    2019 |        9 |     16853 |          8 |
|    2019 |        7 |     14233 |          9 |
|    2019 |        4 |     13221 |         10 |
|    2019 |        6 |     12658 |         11 |
|    2019 |        5 |      9964 |         12 |
|    2020 |        1 |     20347 |          1 |
|    2020 |       12 |     18636 |          2 |
|    2020 |        2 |     17434 |          3 |
|    2020 |        3 |     16225 |          4 |
|    2020 |       11 |     15742 |          5 |
```

```
|    2020 |         5 |     14589 |           6 |
|    2020 |         4 |     13853 |           7 |
|    2020 |        10 |     13842 |           8 |
|    2020 |         6 |     13248 |           9 |
|    2020 |         9 |     11467 |          10 |
|    2020 |         8 |      9378 |          11 |
|    2020 |         7 |      8728 |          12 |
+---------+-----------+-----------+-------------+
24 rows in set (0.00 sec)
```

## Exercise 16-3

Write a query that retrieves all 2020 data, and include a column that will contain the `tot_sales` value from the previous month.

```
mysql> SELECT year_no, month_no, tot_sales,
    ->   lag(tot_sales) over (order by month_no) prev_month_sales
    -> FROM sales_fact
    -> WHERE year_no = 2020;
+---------+----------+-----------+------------------+
| year_no | month_no | tot_sales | prev_month_sales |
+---------+----------+-----------+------------------+
|   2020 |       1 |    20347 |         NULL |
|   2020 |       2 |    17434 |        20347 |
|   2020 |       3 |    16225 |        17434 |
|   2020 |       4 |    13853 |        16225 |
|   2020 |       5 |    14589 |        13853 |
|   2020 |       6 |    13248 |        14589 |
|   2020 |       7 |     8728 |        13248 |
|   2020 |       8 |     9378 |         8728 |
|   2020 |       9 |    11467 |         9378 |
|   2020 |      10 |    13842 |        11467 |
|   2020 |      11 |    15742 |        13842 |
|   2020 |      12 |    18636 |        15742 |
+---------+----------+-----------+------------------+
12 rows in set (0.00 sec)
```

Previous chapter

< A. ER Diagram for Example Database

Next chapter

Index >